CQRS — Command Query Responsibility Segregation

Category: Data / messaging **One-liner:** Split a system's operations into commands (writes) and queries (reads) with separate models, so each can be optimised, scaled, and evolved independently.

Part 0 — Concepts you need first

What is CRUD?

Create, Read, Update, Delete — the four basic operations on stored data. Most applications start here: one `User` class, one `users` table, one set of methods that create, read, update, and delete users.

CRUD is simple and intuitive, and for most applications it's the right answer. This chapter is about what happens when it stops being enough.

What is a "model"?

In this chapter, a **model** means the code and data structures representing part of your **domain** (the business area the software deals with) — the `User` class, its **validation** rules (checks that input is acceptable before it's used), its database table, the code that loads and saves it.

CQRS's central claim is that **one model trying to serve both reads and writes ends up serving neither well**, because the two have genuinely opposed requirements.

Reads vs. writes — why they differ

This is the intuition the whole pattern rests on.

	Writes	Reads
Must be	Correct, validated, secure	Fast
Volume	Usually lower	Usually far higher
Complexity	Business rules (domain rules), constraints (database-enforced rules)	Joining (combining tables) and formatting
Cost of error	Severe — corrupt data	Mild — a stale (out-of-date) display
Optimal storage	Normalised (no duplication)	Denormalised (pre-joined)

That last row is the deep conflict. **Normalised** storage stores each fact once — excellent for writes, because updating a fact means changing one row. **Denormalised** storage duplicates data into ready-made shapes — excellent for reads, because the answer is already assembled with no **joins** (SQL operations that combine rows from several tables at query time).

You cannot have both in one schema (the defined structure of tables and columns). Every design is a compromise between them — and CQRS says: stop compromising, have two.

What is a DTO?

A **Data Transfer Object** — a simple object whose only job is to carry data to whoever asked for it. It has no **business logic** (code implementing the rules of the domain). In CQRS, the read side returns DTOs shaped for the screen, rather than returning domain objects.

What does "side effect" mean?

A change to state that persists after a method returns. Reading data has no side effects; saving data does. This distinction is what CQRS formalises.

Part 1 — The problem

In a **monolithic** application (one built and deployed as a single unit) we usually use **one database for everything**. That single database handles both read and write operations — meaning it's simultaneously serving **complex join queries** *and* performing **CUD** (Create, Update, Delete) operations. As the application gets more complicated, **handling these read and write tasks becomes really hard to manage.**

The scale mismatch

In an ideal world, CRUD operations provide a simple and intuitive method for handling data. But complex applications demand handling numerous **concurrent** operations (happening at the same time), diverse data requirements, and efficient scaling — and traditional CRUD struggles to keep up.

The e-commerce picture: thousands of users browsing products (**read** operations) while concurrently there are inventory updates, order placements, and product additions (**write** operations).

Can a single model efficiently handle these varied requirements? That's where the cracks show.

Note the asymmetry: **read operations are significantly more numerous, while write operations carry more complexity.** Can we scale them independently based on their different needs? Not with a single data model handling both.

The optimisation conflict — the banking example

A banking application handles two main kinds of operation:

- **Transactions** — depositing or withdrawing money
- **Balance inquiries** — checking the current account balance

Their needs are quite different:

Writing data (transactions) involves creating new records or updating existing ones. It needs to be **accurate and secure**, as it directly affects a user's finances.

Reading data (balance inquiries) is about fetching data **quickly and efficiently**. Users expect to see their balance in **real time** (immediately reflecting the latest state), without delay.

> **The trap, stated plainly:** if you focus too much on making transactions fast and secure, you end up making balance inquiries slower. If you optimise for quick balance checks, you compromise the efficiency or security of processing transactions. **One model cannot be tuned for both.**

This is a classic case where the traditional approach falls short. It becomes difficult to maintain, scale, and perform well under different types of **load** (the volume and kind of work hitting the system).

Part 2 — The solution

Break the term down:

- **Command** — an instruction to *do* something: creating, updating, or deleting data. In the banking example, processing a deposit or a withdrawal.
- **Query** — a request to *retrieve* data **without changing it**. Checking your account balance.
- **Responsibility Segregation** — dividing responsibilities: separating the handling of commands (writes) from queries (reads).

The two sides

The Command Side handles all write operations. It's responsible for updating data — processing transactions in the banking app.

The Query Side deals with all read operations. It's optimised for fetching data quickly — showing your current account balance.

The formal rule

This is the definition that gives the pattern its rigour, and it's worth memorising. The **separation of concerns** (giving each piece of code one clear job) distinguishes between methods that change state and those that don't. **Each method of an object falls into one of these categories, but never both:**

	Returns a value?	**Changes state?**
Query	✅ Returns a result	❌ No state change, no side effects
Command *(also called modifiers or mutators — methods that change data)*	❌ Only an indication of success or failure	✅ Changes system state

> **Why "never both" matters.** A method that both changes state *and* returns data is impossible to call safely — you can't retry it (it might apply twice), you can't **cache** it (store its result to reuse later), and you can't reason about it. Enforcing the split makes queries freely repeatable and cacheable, and makes commands explicit about being consequential.

Why this is useful

Optimization — by separating reads and writes you can optimise each independently. The command side focuses on security and accuracy; the query side is optimised for speed.

Simplicity — it simplifies the design. Each side is developed and maintained separately, reducing complexity.

Scalability (handling more load by adding resources) — as the application grows, scale read and write operations independently based on their specific demands.

Flexibility in technology choice — different **technology stacks** (sets of languages, frameworks and databases) can be used for the read side and the write side if that suits their needs better. A **relational database** (one storing data in tables linked by keys, queried with SQL) for writes with its constraints and **transactions** (groups of changes that succeed or fail together); a **document store** (a database storing self-contained JSON-like documents) or **search index** (a structure built for fast text search, e.g. Elasticsearch) for reads.

In essence: CQRS acknowledges that the needs for reading and writing data are different, and lets you treat them separately.

Part 3 — Architecture: three levels

CQRS is not one thing — it's a spectrum. Knowing which level you mean is important, because the costs differ enormously.

Level 1 — Separate models, one database

The Command Model handles all write operations. It is responsible for maintaining the **integrity and consistency** of data (keeping it correct and obeying all its rules). It receives commands, performs validations, applies business rules, and alters system state. *The course's image: a disciplined librarian, ensuring every book is in its proper place.*

The Query Model handles all read operations, providing the current state of the system to users. It's optimised for reads, ensuring users get data efficiently and quickly. *A friendly librarian, always ready to fetch the book you need.*

This is the simplest useful form — two models, one database. **Most teams should stop here.** You get most of the design clarity with almost none of the operational cost.

Level 2 — Separate read and write databases

To enhance flexibility and scalability, you can use **separate databases** for reads and writes.

> ⚠️ However, this **significantly increases complexity**, as it requires careful consideration of how to propagate changes from the write database to the read database.

The specific problems this creates, all of which the course lists:

- **No atomic transactions across the read and write models** (no single all-or-nothing update covering both) — so you must rely on **asynchronous messaging and events** (messages sent without waiting for a reply) to achieve **eventual consistency** (the read side catches up after a short delay — see System Design Fundamentals)
- **Out-of-order event propagation**
- **Event loss**
- **Sync discrepancies**
- **Read Model update failures**
- **The timing of UI updates after writing** (UI = user interface) — what does the user see immediately after they hit save, if the read model hasn't caught up?

Implementing and managing this style **demands a high degree of complexity and a deep understanding of distributed transactions** (transactions spanning several databases or services). It's suitable **only if your application's non-functional requirements** (needs about speed, scale and reliability rather than features) **genuinely demand such an intricate solution.**

Level 3 — Event Sourcing

This exists to solve one specific problem from Level 2: **ordering**.

Using separate databases makes maintaining sync with eventual consistency hard, and **the sequence of events from write side to read side is crucial**. If updates on a write model occur quickly one after another, **receiving events out of order leaves the read model with outdated information.**

The complication: **many asynchronous message buses** (systems that carry messages between components) **prioritise availability** (staying up and answering) **and performance, and often don't guarantee the order of message delivery.**

Event Sourcing addresses this with a different method of storing write models: **append-only event stores** (logs where new events are only ever added, never edited or deleted) that record every action

on a model. When updating, the model is refreshed by **replaying its entire event history**.

In this approach each write model instance exists as an **independent event stream** (its own ordered sequence of events), which enables:

- **Various data views**, by replaying the events differently
- **Rebuilding read models** from scratch by retrieving all events from the write side — the direct fix for synchronisation problems
- **Simplified auditing** (proving later who changed what, and when), since the event streams document every change to every model
- **Creating new read models** by replaying past events — you can add a new view of historical data that never existed before

> **However:** implementing event sourcing **adds complexity and is a significant undertaking**, especially for anyone new to CQRS, event sourcing, or **distributed systems** (systems spread across many machines). Do not adopt Level 3 because you adopted Level 1. They are separate decisions with very different costs.

Part 4 — Inner workings

The journey of a command

A command signifies an **intention to change state**. Follow one through: a user wants to update their address in an e-commerce application. The command is `UpdateUserAddress`.

Step 1 — Validation. Before anything else, the command is validated. This can involve checks for **data format, permissions, and other business rules**. Is the new address in a valid format? Does the user have permission to change it? Pass, and it proceeds; fail, and it's **rejected**.

Step 2 — Handling. Once validated, the command is handled by the **command handler** — the workhorse of the command side. It's responsible for updating system state based on the command; here, updating the user's address.

Step 3 — Generating events. Every command leads to a change in state, represented by an **event** — here, `UserAddressUpdated`. This event is stored in the **event store**, serving as a historical record of all changes in the system.

The query's quest

A query is simply a request for data. Our user now wants to view their updated address, issuing `GetUserAddress`.

Unlike commands, queries require no validation and generate no events. They're simply processed by the **query handler**, which fetches the required data and returns it.

> That asymmetry is worth noticing — it's a direct consequence of the formal rule. Because a query changes nothing, there's nothing to validate against business rules and nothing to record. This is why the read path can be made so much simpler and faster than the write path.

Synchronising the two sides

How do the two sides stay in sync? This is where **Event Sourcing** comes in.

The `UserAddressUpdated` event generated when the user updated their address is **replayed on the query side** to update its state. The user's updated address is thereby reflected in the query model. Event

Sourcing ensures that despite working separately, **the command and query sides remain in sync**.

Part 5 — Worked example: user profile management

A simple application managing user profiles. You can **create a new user** (command), **update their email** (command), and **view user details** (query).

The pieces

Command Model (Write Model):

- **Command Class** — defines an action to perform, e.g. `CreateUserCommand` with `userId`, `name`, `email`
- **Command Handler** — processes commands: takes the command, performs business logic, persists changes
- **Domain Model** — represents the state of the data (the `User` class)
- **Repository** — handles data **persistence** (saving to and loading from storage)

Query Model (Read Model):

- **DTO** — represents the data we want to read, e.g. `UserProfileDTO`
- **Query Class** — defines a query, e.g. `GetUserProfileQuery` with a `userId`
- **Query Handler** — handles the query and returns the data, often using **read-optimised storage**

Infrastructure:

- **Command Bus** — routes commands to their handlers
- **Query Bus** — routes queries to their handlers
- **Event Store** — in a more advanced setup, stores the events resulting from commands

Command classes

```
public class CreateUserCommand {
    private String userId;
    private String name;
    private String email;
}

public class UpdateUserEmailCommand {
    private String userId;
    private String newEmail;
}
```

Note these are **just data** — an intention expressed as an object, with no behaviour. Making the intent an explicit object is part of the point: you can log it, queue it, or replay it.

Command handlers

```
public class CreateUserCommandHandler {
    private UserRepository userRepository;

    public CreateUserCommandHandler(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    public void handle(CreateUserCommand command) {
        User user = new User(command.getUserId(), command.getName(), command.getEmail());
        userRepository.save(user);
    }
}

public class UpdateUserEmailCommandHandler {
    private UserRepository userRepository;

    public UpdateUserEmailCommandHandler(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    public void handle(UpdateUserEmailCommand command) {
        if (!isValidEmail(command.getNewEmail())) {
            throw new IllegalArgumentException("Invalid email format");
        }
        User user = userRepository.findById(command.getUserId());
        if (user != null) {
            user.setEmail(command.getNewEmail());
            userRepository.save(user);
        }
    }

    private boolean isValidEmail(String email) {
        return email != null && email.contains("@");
    }
}
```

Two things to notice. First, `handle` returns `void` (Java for "returns nothing") — matching the formal rule that commands don't return data. Second, **validation lives here**, in the command handler, before any state change. This is the "accuracy and security" focus of the write side made concrete.

Domain model and repository

```
public class User {
    private String id;
    private String name;
    private String email;
}

public interface UserRepository {
    void save(User user);
    User findById(String id);
}
```

The read side

```
public class UserProfileDTO {
    private String id;
    private String name;
    private String email;
}

public class GetUserProfileQuery { private String userId; }
public class ReadUserEmailQuery  { private String userId; }
```

```
public class GetUserProfileQueryHandler {
    private UserRepository userRepository;

    public UserProfileDTO handle(GetUserProfileQuery query) {
        User user = userRepository.findById(query.getUserId());
        return new UserProfileDTO(user.getId(), user.getName(), user.getEmail());
    }
}

public class ReadUserEmailQueryHandler {
    private UserRepository userRepository;

    public String handle(ReadUserEmailQuery query) {
        User user = userRepository.findById(query.getUserId());
        return user != null ? user.getEmail() : null;
    }
}
```

Notice the granularity. `ReadUserEmailQuery` returns just a `String` — not a whole object. Read models can be **as narrow as the screen actually needs**, and that's precisely the optimisation opportunity CQRS opens up. Each query gets its own purpose-built shape rather than everyone sharing one bloated object.

Also note the handlers return **DTOs, not domain objects**. The read side is deliberately not exposing the write side's model.

Infrastructure

```
public class SimpleCommandBus implements CommandBus {
    private CreateUserCommandHandler handler;

    @Override
    public void dispatch(CreateUserCommand command) {
        handler.handle(command);
    }
}

public class SimpleQueryBus implements QueryBus {
    private GetUserProfileQueryHandler handler;

    @Override
    public UserProfileDTO dispatch(GetUserProfileQuery query) {
        return handler.handle(query);
    }
}

public class SimpleEventStore implements EventStore {
    private List<UserEvent> events = new ArrayList<>();

    @Override
    public void storeEvent(UserEvent event) {
        events.add(event);
    }
}
```

The buses route to handlers; the event store captures changes as a series of events which, in a full application, can be **replayed to rebuild the state of an entity** (one individual business object, such as a specific user).

> **Where this example actually sits:** it's **Level 1**. Both sides still share the same `UserRepository` and the same database. The event store is present but unused by the read path. That's not a flaw — Level 1 is where most real systems should live, and it already delivers the design clarity that is CQRS's main benefit.

Part 6 — Issues, considerations, and performance

Complexity

Implementing CQRS introduces **a fair amount of complexity**. Maintaining two separate models — command and query — is a complex task. This can be a roadblock for developers unfamiliar with the pattern, and **the learning curve is steep**.

> The course's image: like weaving a basket, the more intricate the design, the more complex the weaving process.

Managing data consistency

One of the significant challenges is maintaining consistency between the command model and the query model.

The scenario: the command model changes, but the query model hasn't been updated yet. **What happens if a query arrives during that interval? It returns outdated data.**

> This is the central trade-off of CQRS, and the one to raise in an interview. In UI terms: a user saves their profile, the page reloads, and their old name is still showing. Common mitigations include having the UI **optimistically** display (show the expected result before the server confirms it) what the user just submitted, or routing reads immediately after a write to the write model.

Handling transactions

In traditional CRUD, handling transactions is relatively straightforward. In CQRS it gets complicated.

The scenario: you need to perform multiple commands as part of a single logical transaction. What if one command fails? **How do you roll back the changes from previous commands?** Remember, commands change system state.

> The course's image: like painting a wall — if you make a mistake with one colour you can't simply erase it; you may need to repaint the entire wall.

(This is exactly the problem the Saga pattern addresses with compensating transactions.)

Dealing with asynchronicity

CQRS often comes with asynchronous operations, making **error handling and debugging more complex**. In a system where operations are performed asynchronously, **tracking down the source of an error is challenging**.

The performance upside

On the brighter side, a compelling reason to use CQRS is its **potential to improve performance**. By segregating commands and queries you optimise each side for its specific purpose.

For instance, if you find queries outnumber commands, you can **allocate more resources to the query side** to ensure smooth operation.

> The course's image: putting more trains on your busiest routes while maintaining fewer services where demand is low.

Part 7 — Use cases

High-performance applications

The most compelling case: applications where **the number of reads significantly outweighs the number of writes**. An e-commerce platform where millions of users browse products (queries) compared to a much smaller number of transactions (commands). It makes obvious sense to optimise the system for the higher volume of reads.

Complex business logic

Where a system has **complex business logic that modifies state**, separating read and write responsibilities simplifies that complexity.

> The course's image: division of labour on a factory assembly line — each worker focuses on their specific task, leading to improved efficiency and higher quality output.

The practical benefit: your write model can be rich with rules and **invariants** (conditions that must always hold, e.g. "balance never negative") without that complexity slowing down or complicating every read.

Collaborative domains

Where **many users can update the same data**, CQRS helps manage conflicts. Picture a shared document edited by several people simultaneously. **If updates are treated as commands that can fail or be rejected based on rules**, it becomes much easier to handle this collaborative environment.

Part 8 — System design examples

Event-driven microservices

An e-commerce system handling orders. Each order goes through stages: **placed, paid, shipped, delivered**.

Each stage is a **state change driven by commands**. Meanwhile, different **microservices** (small, independently deployed services) need to **query** the state of an order — a shipment tracking service, a customer notification service. Using CQRS here leads to a cleaner, more efficient system design.

Real-time analytics

(**Analytics** means computing statistics and reports from raw data.) A social media platform where users are constantly **posting updates (commands)** while the platform simultaneously provides **real-time analytics about user engagement (queries)**.

Their performance needs are genuinely opposite:

- **Commands must be fast and reliable** to provide a smooth user experience
- **Queries require heavy computation** but are **not time-critical**

By segregating these responsibilities, the system can **optimise resource allocation to meet these differing needs**. This is a textbook fit — the two **workloads** (streams of work of one kind) would actively harm each other sharing one model.

Part 9 — Common beginner misconceptions

"CQRS requires two databases." No — that's Level 2. Level 1 is two models over one database, and it's where most systems should live.

"CQRS requires event sourcing." No. They're frequently discussed together and complement each other, but they're independent choices. You can do CQRS with no events at all.

"CQRS is the same as read replicas." Read replicas are copies of the *same* database, kept in sync, used to spread read **load**. CQRS means a *different model* shaped for reading. You can use both.

"Commands are just HTTP POSTs." (A POST is the web request type used to create or submit something.) Commands are a design concept, not a **transport** detail (how the message travels). A command is an explicit object representing intent — which is what lets you queue, log, and replay it.

"It's always faster." It adds **indirection** (extra layers between caller and data). It's faster when read and write workloads genuinely conflict, and pure overhead when they don't.

"Apply it to the whole application." Almost always wrong. Apply it to the **specific parts** where read and write needs genuinely diverge, and leave simple CRUD as CRUD.

Part 10 — Check your understanding

1. Why can't one database schema be optimal for both reads and writes?
2. What are the two halves of the formal rule, and why does "never both" matter?
3. Which level of CQRS should most teams adopt, and why?
4. What specific problem does event sourcing solve that Level 2 has?
5. What is the central trade-off, expressed as something a user would notice?
6. Why is the social media analytics example a textbook fit?

(Answers: 1 — writes want normalised storage where each fact lives once, reads want denormalised pre-joined shapes, and these are directly opposed; 2 — queries return data and change nothing, commands change state and return only success or failure; "never both" makes queries safely repeatable and cacheable and makes commands explicitly consequential; 3 — Level 1, two models over one database, because it gives the design clarity with almost none of the operational cost; 4 — out-of-order event delivery leaving the read model stale, solved by replaying an append-only event stream; 5 — the consistency window: a user saves their profile and the page still shows the old value; 6 — posting must be fast and reliable while analytics is computation-heavy but not time-critical, so the two workloads would degrade each other if they shared a model.)

Part 11 — Interview soundbite (30 seconds)

> "CQRS separates the write path from the read path — commands that change state and return nothing, queries that return data and change nothing. The motivation is that reads and writes usually have opposite requirements: in a banking app, transactions need accuracy and security while balance inquiries need raw speed, and at the storage level writes want normalised data while reads want denormalised pre-joined shapes, so you can't tune one model for both. It's a spectrum, and being clear about which level you mean matters. The simplest version is two models over one database, which is where most systems should stop — you get the design clarity with almost no operational cost. The next level is separate read and write databases, which buys independent scaling but costs you atomic transactions, so you're on eventual consistency via async events and dealing with out-of-order delivery and lost messages. The furthest level is event sourcing, where the write side is an append-only event log and you rebuild read models by replaying it. That solves ordering and gives you audit history and the ability to build entirely new read models from past events, but it's a big undertaking and a separate decision from adopting CQRS. The main downside throughout is the consistency window — a user saves their profile and the page still shows the old name — plus genuine complexity and harder debugging. It pairs naturally with event-driven architecture, and with sagas when one logical transaction spans multiple commands."

Part 12 — Implementation

A. Runnable demo

Full source: `demos/CqrsDemo.java`

```
cd demos && ./gradlew runCh12
```

The formal rule made mechanical — command handlers return `void`:

```
void handle(ChangeEmail cmd) {                      // <- returns void
    User u = writeStore.get(cmd.userId());
    if (!cmd.newEmail().contains("@")) throw new IllegalArgumentException("invalid email");
    String old = u.email;
    u.email = cmd.newEmail();
    publish(new UserEvent.EmailChanged(cmd.userId(), old, cmd.newEmail()));
}

UserProfileView get(String id) { return store.get(id); }   // <- returns data, changes nothing
```

Rebuilding a read model from the event log — the Level 3 superpower:

```
fast.clear();                               // wipe the read model entirely
eventStore.all().forEach(fast::apply);      // replay history
```

B. What you should see

```
=== 3. Level 2: separate read store -> CONSISTENCY WINDOW ===
  write store immediately: carol@new.com
  read model immediately:  carol@old.com   <- STALE
  read model after 300ms:  carol@new.com   <- converged

=== 4. Several read models from ONE write ===
  profile view : UserProfileView[id=u4, name=Dave, email=dave@x.com]
  directory view: [UserDirectoryEntry[name=Dave, initial=D], ...]

=== 5. Level 3: rebuild the read model by replaying events ===
  after wiping the read model:  null
  after replaying the log:      UserProfileView[id=u1, name=Alice, ...]
```

Section 3 is the trade-off you must be able to describe in an interview, and here it's observable: the write store has the new email while the read model still has the old one. That's not a bug — it's the deal you accepted. Section 5 shows the compensating power: **read models are disposable**, because the event log is the truth.

C. Production — Axon Framework

(**Axon** is a Java framework for CQRS and event sourcing. An **aggregate** is a cluster of objects treated as one unit for changes — here, one user — and annotations like `@CommandHandler` tell Axon which method handles which message.)

```java
@Aggregate
public class UserAggregate {

    @AggregateIdentifier private String userId;
    private String email;

    @CommandHandler
    public UserAggregate(CreateUserCommand cmd) {
        if (!cmd.email().contains("@")) throw new IllegalArgumentException("invalid email");
        AggregateLifecycle.apply(new UserCreatedEvent(cmd.userId(), cmd.name(), cmd.email()));
    }

    @CommandHandler
    public void handle(ChangeEmailCommand cmd) {
        if (cmd.newEmail().equals(this.email)) return;              // no-op, no event
        AggregateLifecycle.apply(new EmailChangedEvent(cmd.userId(), this.email, cmd.newEmail()));
    }

    // State is derived by applying events - never assigned directly
    @EventSourcingHandler
    public void on(UserCreatedEvent e) { this.userId = e.userId(); this.email = e.email(); }

    @EventSourcingHandler
    public void on(EmailChangedEvent e) { this.email = e.newEmail(); }
}
```

The projection — the read side: (a **projection** builds a read model by applying events to it)

```java
@Component
public class UserProfileProjection {

    private final UserProfileRepository repo;

    @EventHandler
    public void on(UserCreatedEvent e) {
        repo.save(new UserProfileView(e.userId(), e.name(), e.email()));
    }

    @EventHandler
    public void on(EmailChangedEvent e) {
        repo.findById(e.userId()).ifPresent(v -> { v.setEmail(e.newEmail()); repo.save(v); });
    }

    @QueryHandler
    public UserProfileView handle(FindUserProfileQuery q) {
        return repo.findById(q.userId()).orElseThrow();
    }
}
```

Plain Spring, no framework — Level 1, which is where most systems should live. (**Spring** is the main Java application framework; **JPA**, the Java Persistence API, maps Java classes to database tables; a **native query** is raw SQL written by hand.)

```java
// Write side: JPA against a normalised schema
public interface UserRepository extends JpaRepository<User, String> {}

// Read side: a projection interface or native query against a denormalised view
public interface UserProfileReadRepository extends Repository<UserProfileView, String> {
    @Query(value = """
        SELECT u.id, u.name, u.email, COUNT(o.id) AS order_count
        FROM users u LEFT JOIN orders o ON o.user_id = u.id
        WHERE u.id = :id GROUP BY u.id
        """, nativeQuery = true)
    UserProfileView findProfile(@Param("id") String id);
}
```

> **You do not need Axon, Kafka, or event sourcing to do CQRS.** Two repositories and two model classes over one database is legitimate CQRS and delivers most of the benefit.

Handling the consistency window in the UI — three practical options:

```java
// 1. Read-your-own-writes: route reads right after a write to the write model
if (recentlyWrittenBy(userId)) return writeModel.find(userId);
return readModel.find(userId);

// 2. Return the new state from the command endpoint so the UI can render optimistically
// 3. Return a version token; the client polls the read model until it catches up
```

(In option 3, a **version token** is a number identifying the write; to **poll** is to ask repeatedly until the answer is ready.)

D. Implementation pitfalls

1. Jumping straight to Level 3. Event sourcing is a large, hard commitment — schema evolution for events, **snapshotting** for long streams (saving the full state every so often so replay can start from there), replay tooling. Adopting it *because* you adopted CQRS is the most common way teams get into trouble. Start at Level 1.

2. Commands that return data. The moment `handle()` returns the updated entity, the rule collapses and you're back to one model. Return void, or at most an ID.

3. Business logic in the read model. Projections should only *shape* data. Any rule that decides something belongs on the write side, or the two sides will disagree.

4. Ignoring the consistency window in the UI. Users who save and immediately see stale data file bugs. Design for it explicitly with one of the three techniques above.

5. Not being able to rebuild projections. The main benefit of event sourcing is disposable read models — but only if replay is *tested*. Make rebuilding a routine operation, not a theoretical one.

6. Unbounded event streams. An aggregate with 100,000 events takes forever to load by replay. Use **snapshots** — persist state every N events and replay only from there.

7. Applying CQRS everywhere. Simple CRUD entities should stay CRUD. Apply it where read and write needs genuinely diverge, and be able to say *why* for each place you use it.

8. Events named as commands. `UpdateEmail` is a command; `EmailChanged` is an event. Getting the tense wrong signals a design where events are really disguised instructions, which reintroduces **coupling** (components depending on each other's internals).

Related patterns

- **Event-Driven Architecture** — the substrate that syncs command and query sides
- **Saga** — for transactions spanning multiple commands, where rollback isn't automatic
- **BFF** — read models shaped per client is the same instinct as BFF applied to data